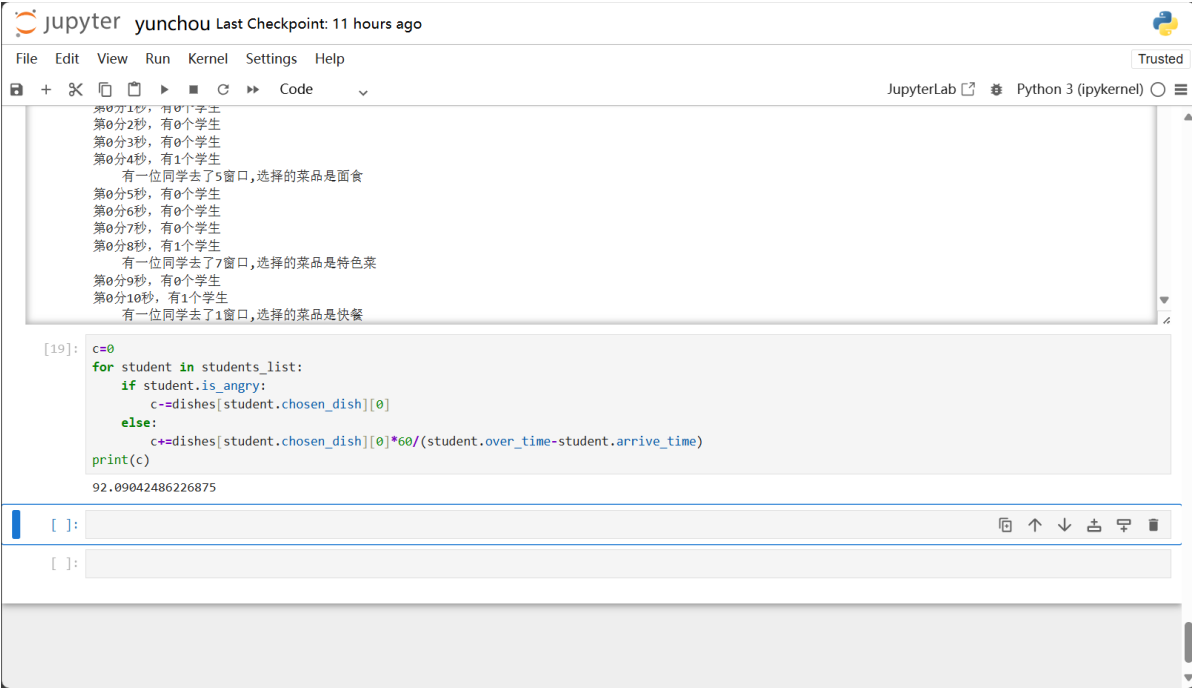
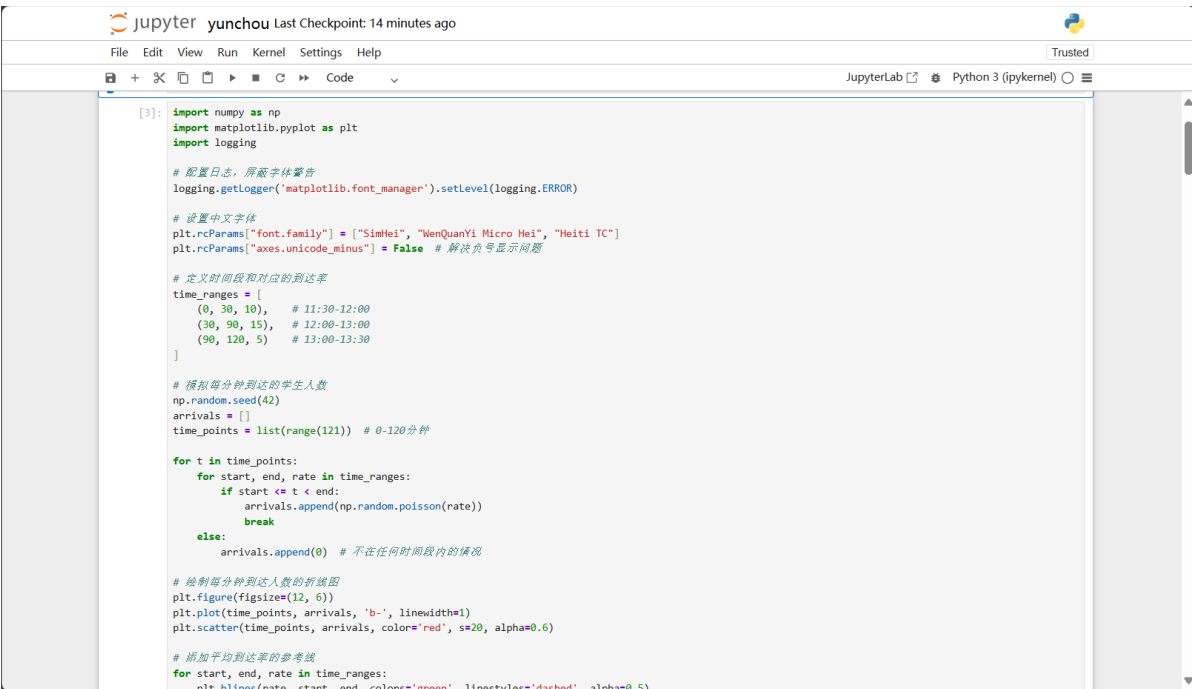


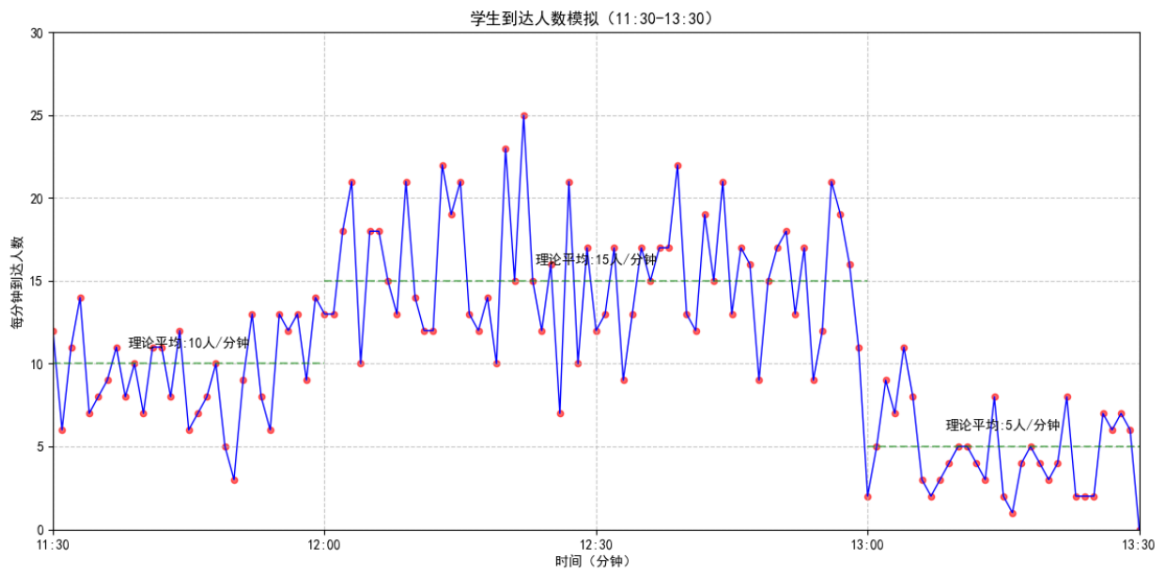
代码说明文档

按照题意以及一系列模拟显示的规则（见另一文档），模拟了一下最优解的真实情况，基本上可以起到一个**验证答案**的作用，如果验证时用纯数学求前向解，一是太难，二是太理想化，丧失了随机性，不够直观。这个模拟程序按秒遍历，相当于**按照给定的窗口分布将整个时间段的完整过程模拟了一遍，并最终求出总的满意度**。可以输出**每秒上来了多少个学生，分别选择了多少个窗口**。而且还可以**动态输出每秒每个窗口的排队人数的柱状图**，和**累积到这一秒的总满意度的折线图**，最后输出的c是总的满意度（因为随机性每次重新跑值都会不一样），满意度的求法多加了一条如果学生无队可排，学生就会愤怒离去，并且他的**满意度变为负的他所选菜品对应的热门度**



对如下这一段，通过泊松过程根据对应时间段的平均人流量生成了每分钟到达的人数。





对如下这段

```
[8]: from collections import OrderedDict
import random
total_seconds=120*60 # 7200秒
second_dict=OrderedDict([(second,0) for second in range(total_seconds)])

# 随机分配人数到各秒
for minute in range(120):
    minute_people=arrivals[minute]
    start_second=minute * 60

    # 为每个人随机分配秒数 (0-59, 对应start_second到start_second+59)
    for _ in range(minute_people):
        random_second=random.randint(0, 59)
        total_second=start_second+random_second
        second_dict[total_second]+=1
for i in range(7200,7260):#为了保证全部学生都能取到菜, 将食堂开放时间延长一分钟
    second_dict[i]=0

7260
```

是将每分钟内到达的总人数随机分配到这分钟内的若干秒上, 最后补了一分钟的0, 是为了保证每个学生都能完整的取到餐, 如果只遍历到7200s, 那么最后一分钟来排队或者排长队的学生会取不到餐。

对如下这段, 是对窗口和学生两个类的定义,

- 对窗口定义了其编号, 是否开放, 提供菜品, 队伍列表, 是否在做菜, 做菜剩下的时间, 改菜品的标志。
- 对学生定义了其到达时间, 离开时间, 选择的菜品, 选择的窗口, 是否愤怒离去

```
[10]: class Window:
    def __init__(self, OpenOrClose, dishname, sign):
        self.sign=sign#窗口编号, 从一开始
        self.open=OpenOrClose
        self.dish=dishname
        self.queue_num=0
        self.queue=[]
        self.iscooking=False
        self.cooktimeleft=0
        self.change_flag=0

    def close(self):
        self.open=False
        self.iscooking=False
        self.queue_num=0

    def checkiscooking(self):
        if self.queue_num==0:
            self.iscooking=False
            return False
        return True

    def change(self, openstatus, dishname):
        self.open=openstatus
        self.dish=dishname
        self.change_flag=0
```

```
[11]: class Student:
    def __init__(self, arrive_time):
        self.arrive_time=arrive_time
        self.over_time=0
        self.chosen_times=0
        self.chosen_dish=None
        self.chosen_window=0
        self.is_angry=False
```

对如下这又臭又长一张图截不完的这段，是学生选择窗口的函数，确实能用。具体选择规则见另一个文档

```
[12]: options=["快餐", "面食", "特色菜"]
weights=[0.2, 0.3, 0.5]
def choose_window(student, windows_list, students_list):
    choice=random.choices(options, weights=weights, k=1)[0]#按照权重随机选择一个
    student.chosen_dish=choice
    window_signs=[]#提供这个菜品的窗口
    window_queue_num=[]#对应排队人数
    change_flag=0#是否改变选择的标志
    for window in windows_list:
        if window.open==False:#如果没开门不管他
            continue
        if window.dish==choice:#如果提供对应菜品
            window_signs.append(window.sign)#记录对应的窗口号与排队人数
            window_queue_num.append(len(window.queue))
    if len(window_signs)==0:
        change_flag=1
        student.chosen_times+=1
    if len(window_signs)!=0:#如果存在卖他的窗口
        queue_num_min=min(window_queue_num)#找到排队人数最少的窗口号
        sign_min=window_signs[window_queue_num.index(queue_num_min)]
        if queue_num_min<10:#如果最小值小于10, 就排这一条
            student.chosen_window=sign_min
            windows_list[sign_min-1].queue.append(student)
            windows_list[sign_min-1].queue_num+=1
        if queue_num_min==10:#如果最小值为10, 说明全部排满了
            change_flag=1
            student.chosen_times+=1

    if change_flag==1:#如果确实发生了第二次选择
        change_flag=0
        options_tool=["快餐", "面食", "特色菜"]
        weights_tool=[0.2, 0.3, 0.5]
        index_tool=options_tool.index(choice)#在剩下的两个菜中选
        del options_tool[index_tool]
        del weights_tool[index_tool]
        choice=random.choices(options_tool, weights=weights_tool, k=1)[0]
        student.chosen_dish=choice
```

对如下这段，是每个窗口在三个时间段内的开闭情况与提供的菜品情况，如果想模拟其他策略下的情况，直接改这个数组即可

```

: windows_set=[
    [(True,"快餐"),(True,"快餐"),(True,"快餐"),(True,"快餐"),(True,"面食"),(True,"面食"),(True,"特色菜"),(True,"特色菜"),(False,""),(False,"")],
    [(True,"快餐"),(True,"快餐"),(True,"快餐"),(True,"快餐"),(True,"面食"),(True,"面食"),(True,"特色菜"),(True,"特色菜"),(True,"快餐"),(True,"特色菜")],
    [(True,"快餐"),(True,"快餐"),(True,"快餐"),(False,""),(True,"面食"),(True,"面食"),(True,"面食"),(False,""),(False,""),(False,"")]
]
print(windows_set[0][0][1])
快餐

```

下面是主循环，虽然写的史山，确实能够跑起来。最下面的

```

plt.clf() # 清除之前画的图
plt.plot(x2,y2) # 画出当前x列表和y列表中的值的图形
plt.pause(0.001) # 暂停一段时间，不然画的太快会卡住显示不出来

```

这三行是绘制的 (x2, y2) 的图，即每秒的累计满意度的折线图，横坐标为秒数，纵坐标是对应的累计的满意度；如果将这三行放在

```

x1=[1,2,3,4,5,6,7,8,9,10]
y1=[]
for window in windows_list:
    y1.append(len(window.queue))

```

这一段的下方，并且将最下面那三行删去，运行时绘制的图是每秒每个窗口对应的排队人数的柱状图。

这些绘制的图像都是可以随着遍历实时变化的，但是因为matplotlib的ion模式不支持输出多张动态图像，所以每次只能看一张图的情况。实际要遍历整整7260次十分的漫长，因此发的视频里只录了一小段。

```

windows_list=[]#初始化
students_list=[]
for i in range(10):
    window=window(windows_set[0][i][0],windows_set[0][i][1],i+1)
    windows_list.append(window)
print(windows_list)

import matplotlib
matplotlib.use('QtAgg')
from matplotlib import pyplot as plt
import numpy as np
%matplotlib qt5
x2=[]
y2=[]
plt.ion()
for i in range(7260):
    '''首先更新所有窗口的状态'''
    for window in windows_list:
        if i==1800:
            window.change_flag=1
        if i==5400:
            window.change_flag=2
        if (not window.iscooking):#对于没有在做菜的窗口
            if window.change_flag!=0:#如果到了改菜品的时候
                window.change(windows_set[window.change_flag][window.sign-1]
[0],windows_set[window.change_flag][window.sign-1][1])#更改的菜品以及窗口开关
                if (window.dish=="特色菜" and special_dish_store==0):#仅针对换菜事
件，如果换的是特色菜但是没库存了，关闭窗口
                window.open=False

```

```

        window.iscooking=False
        window.queue_num=0
    if window.iscooking:#做菜的窗口,菜的剩余时间-1
        window.cooktimeleft-=1
        if window.cooktimeleft==0:#如果时间剩余0,即完成了菜,那么队列排队人数-1
            student_tool=window.queue.pop(0)
            window.queue_num-=1
            student_tool.over_time=i#队首的学生记录结束时间
            student_tool.chosen_dish=window.dish#记录最后拿到的菜品是window此时正
在卖的菜品
            students_list.append(student_tool)#完成的学生存入另一个列表中用于计算满
意度

        if window.change_flag!=0:#如果完成后到了改菜品的时候了
            window.change(windows_set[window.change_flag][window.sign-1]
[0],windows_set[window.change_flag][window.sign-1][1])#更改的菜品以及窗口开关
            if (window.dish=="特色菜" and special_dish_store==0):#仅针对换菜
事件,如果换的是特色菜但是没库存了,关闭窗口
                window.open=False
                window.iscooking=False
                window.queue_num=0
            if window.open==False:#如果关窗口了
                for student in window.queue:#原队列所有人重新选择
                    choose_window(student,windows_list,students_list)
                window.queue=[]

        if len(window.queue)==0:#检查走后是否还剩下人
            window.iscooking=False
        else:
            if window.dish=="特色菜":#检查窗口是否提供特色菜
                if special_dish_store!=0:#如果是而且库存没空
                    window.cooktimeleft=dishes[window.dish][1]#重置做饭时间
                    special_dish_store-=1#减一份库存
                    if special_dish_store==0:#如果库存空了
                        window.open=False
                        window.iscooking=False
                        window.queue_num=0#关窗口
                        for student in window.queue:
                            choose_window(student,windows_list,students_list)
                        window.queue=[]
                    if window.dish!="特色菜":#非特色菜
                        window.cooktimeleft=dishes[window.dish][1]#重置做饭时间
'''再更新新学生'''
if second_dict[i]==0:
    print(f"第{int(i/60)}分{int(i%60)}秒,有0个学生")
if second_dict[i]!=0:#如果有新客人来
    print(f"第{int(i/60)}分{int(i%60)}秒,有{second_dict[i]}个学生")
    for _ in range(second_dict[i]):
        student=Student(i)
        choose_window(student,windows_list,students_list)
        if student.is_angry:
            print("    有一位学生感到愤怒!")
        print(f"    有一位同学去了{student.chosen_window}窗口,选择的菜品是
{student.chosen_dish}")

```

```

x1=[1,2,3,4,5,6,7,8,9,10]
y1=[]
for window in windows_list:
    y1.append(len(window.queue))

for window in windows_list:#针对新上人的窗口
    if window.open:
        if (not window.iscooking and len(window.queue)!=0):
            window.iscooking=True
            if window.dish=="特色菜":#对特色菜窗口
                if special_dish_store==0:#如果库存为0关门，队伍重新选窗口
                    window.open=False
                    window.iscooking=False
                    window.queue_num=0
                    for j in range(len(window.queue)):
                        choose_window(window.queue[j],windows_list,students_list)
                    window.queue=[]
                if special_dish_store!=0:
                    special_dish_store-=1
                    window.cooktimeleft=dishes[window.dish][1]#重置做饭时间
                else:
                    window.cooktimeleft=dishes[window.dish][1]#重置做饭时间
            c=0
            for student in students_list:
                if student.is_angry:
                    c-=dishes[student.chosen_dish][0]
                else:
                    c+=dishes[student.chosen_dish][0]*60/(student.over_time-
student.arrive_time)
            x2.append(i)
            y2.append(c)
            plt.clf() # 清除之前画的图
            plt.plot(x2,y2) # 画出当前x列表和y列表中的值的图形
            plt.pause(0.001) # 暂停一段时间，不然画的太快会卡住显示不出来

```

